

Report: AI Support Agent

Problem Framing

The goal of this project is to build an AI-assisted customer support agent for a brand using real customer-support conversations from the Customer Support on Twitter dataset.

For this project, I selected “AppleSupport” brand from the Customer Support on Twitter dataset because it provides a large volume of customer-support interactions across a diverse range of technical and account-related issues.

The system is designed around three decisions:

Intent → Evidence → Action

Intent: Analyzing what type of problem is the customer reporting and mapping it to an intent.

Evidence: How has Apple Support historically responded to similar customer messages?

Action: Is there sufficient evidence and confidence to draft an automated response, or should the request be escalated to a human?

The goal is not to replace human support agents, but to provide a practical support-assistance pipeline that uses historical support behavior as evidence for automated responses.

What “good” means

For this system, a good support agent should:

- Identify the customer's primary issue consistently and categorize the issue to one of the intent.
- Retrieve historically relevant AppleSupport responses to the problem.
- Produce concise and professional responses grounded in those historical interactions.
- Avoid inventing policies, guarantees, actions, or unsupported troubleshooting instructions.
- Escalate cases to human when the available evidence is insufficient or the model is uncertain.

What I chose not to build

I intentionally scoped the system to the three decisions required by the assignment: **intent classification, evidence-based reply generation, and escalation**. I spent the time building baselines, evaluation, failure analysis, and human/LLM comparison. Several potentially useful capabilities were deliberately left out to keep the implementation focused and evaluable.

- **Multi-intent classification:** Each message is assigned one primary intent. I did not build multi-label classification, although real support messages can contain multiple issues.
- **Conversation-level context:** Each incoming message is processed independently. I did not build persistent conversation memory or use previous turns beyond the retrieved historical examples.
- **Automated support actions:** The agent only drafts a response and recommends whether to escalate. It does not actually reset accounts, process refunds, modify orders, book repairs, or perform other customer-support actions.
- **Real-time support-system integration:** I did not integrate the agent with Apple's internal systems, ticketing platforms, customer accounts, or live support channels.
- **Production deployment and monitoring:** I focused on a reproducible evaluation pipeline rather than production infrastructure, monitoring, authentication, or deployment.
- **Advanced semantic retrieval:** The current system uses TF-IDF and cosine similarity rather than implementing an embedding/vector-database retrieval system. This keeps the baseline transparent and makes its limitations easier to evaluate.

Approach

The system is designed as a sequential pipeline:

Customer message → Intent → Historical evidence → Draft reply → Escalation decision

Intent Classification

A 12-intent taxonomy was derived from recurring AppleSupport customer issues. A manually labeled set of 181 examples was created for classifier development and evaluation. The classifier was evaluated using 5-fold stratified cross-validation, where each fold was evaluated on examples not used for training in that fold. The final classifier was then trained on all 181 labeled examples.

TF-IDF represents each customer message using word and word-pair features, while Logistic Regression predicts the most likely intent. The model also produces a confidence score, which is used as one input to the escalation policy.

Historical Evidence Retrieval

The customer's message is converted into a TF-IDF representation and compared with historical AppleSupport customer messages using cosine similarity.

The top 3 most similar historical conversations are retrieved.

Each retrieved example contains:

- Historical customer message
- AppleSupport response
- Similarity score

These examples provide evidence for the response-generation stage.

Response Generation

The predicted intent and retrieved historical conversations are passed to the openai/gpt-oss-20b model through the Groq API. The model is instructed to draft a concise, professional customer-facing response based on the retrieved historical examples and to avoid inventing policies, guarantees, actions, or unsupported information.

Escalation Policy

The system does not delegate the escalation decision entirely to the LLM. Instead, it uses an explicit rule based on two signals:

- Intent-classifier confidence
- Highest historical retrieval similarity

Strong historical evidence can allow automated handling even when classifier confidence is lower. When both confidence and available historical evidence are insufficient, the request is escalated to a human with an explicit reason.

Evaluation Setup

I sampled 200 direct AppleSupport customer-response pairs using `random_state=42` for reproducibility. I manually assigned each customer message to one of 12 intent categories and excluded messages that were too vague to label reliably, resulting in 181 labeled examples.

The classifier was evaluated using 5-fold stratified cross-validation: each fold trained on approximately 80% of the examples and evaluated on the remaining 20%. The final production classifier was then trained on all 181 examples.

The class distribution remains imbalanced because it reflects the sampled support traffic, with some intents having only 3 - 6 examples.

Evaluation Strategy

I evaluated the system at multiple levels rather than relying on a single metric.

Intent Classification - Majority-class and TF-IDF + Logistic Regression baselines, Accuracy, Macro F1, Weighted F1 and 5-fold stratified cross-validation.

Historical Retrieval

- Retrieve the 3 most similar AppleSupport conversations using TF-IDF + cosine similarity.
- Evaluate similarity across the 181 labeled examples.

Reply Generation & Evaluation

- The LLM generates a reply using the predicted intent and 3 similar historical conversations.
- For evaluation, I used the same openai model as an LLM judge, with a separate prompt and scoring rubric.
- I tested the evaluation on 30 sampled examples, of which 23 successfully completed the judging process.

- The judge scored each reply on groundedness, helpfulness, professionalism, and overall quality.
- I also manually scored the same 23 examples to compare human and LLM-judge agreement.

Human–LLM Agreement

- I manually scored the same 23 replies that were scored by the LLM judge.
- Compared human and LLM-judge scores using exact agreement, agreement within ± 1 , and Spearman correlation.
- I checked how often the scores were exactly the same, within 1 point, and how strongly the two sets of scores were related using Spearman correlation.
- This helped check whether the LLM judge was giving scores reasonably similar to human judgment.

Results

Intent Classification

The majority-class baseline gives 35.14% accuracy and 4.73% Macro F1. It simply predicts ios_update_issue for every message.

The TF-IDF + Logistic Regression classifier gives:

- Accuracy: $38.68\% \pm 7.47\%$
- Macro F1: $23.94\% \pm 4.18\%$
- Weighted F1: $35.32\% \pm 6.82\%$

The classifier performs better than the majority baseline, but the improvement is small. The main limitation is the small and imbalanced labeled dataset, with some intents having only a few examples. This makes minority-intent performance less reliable.

Historical Retrieval

For every customer message, I retrieve the 3 most similar AppleSupport conversations from the historical data using TF-IDF and cosine similarity.

Across the 181 manually labeled evaluation examples, the average similarity of the best match was 0.3636, with a median of 0.3166.

Best Match Similarity	Examples
≥ 0.20	93.4%
≥ 0.30	55.8%
≥ 0.40	25.4%
≥ 0.50	15.5%
≥ 0.60	9.9%

These values measure how closely the messages match, not whether the retrieved answer is correct. Short or generic messages can sometimes receive a high similarity score without providing useful resolution evidence.

Exact copies of the evaluation messages were removed before calculating these retrieval results.

Reply Generation

I used openai/gpt-oss-20b to generate customer-facing replies using the predicted intent and the retrieved historical conversations.

I then used the same model separately as an LLM judge, with a scoring prompt, to evaluate the generated replies.

I tested 30 examples. 23 completed the full generation and judging process, and those 23 were used for the reported scores.

Aspect	Score / 5
Groundedness	4.87
Helpfulness	3.70
Professionalism	4.30
Overall quality	3.87

The replies scored highest on groundedness and professionalism. Helpfulness was lower, which is consistent with cases where the retrieved historical evidence was limited.

Human–LLM Judge Agreement

I manually scored the same 23 replies and compared my scores with the LLM judge's scores.

Aspect	Spearman	Exact match	Within ± 1
Groundedness	0.435	69.6%	82.6%
Helpfulness	0.056	60.9%	69.6%
Professionalism	0.446	73.9%	100.0%
Overall quality	-0.102	60.9%	73.9%

The LLM judge was closer to my manual scores for groundedness and professionalism. The agreement was much weaker for helpfulness and overall quality, so I would use the judge as an evaluation aid rather than treating its scores as ground truth.

Baselines

I used two simple baselines to evaluate the intent classification component.

Baseline 1: Majority Class

Every message is assigned the most common intent, `ios_update_issue`.

- **Accuracy:** 35.14%
- **Macro F1:** 4.73%

Baseline 2: TF-IDF + Logistic Regression

The customer message is converted into TF-IDF word and word-pair features and classified using Logistic Regression with balanced class weights.

- **Mean 5-fold Accuracy:** 38.68%
- **Mean Macro F1:** 23.94%
- **Mean Weighted F1:** 35.32%

The classifier improves over the majority baseline, but the improvement is small. The main reason is the **small and imbalanced labeled dataset**, particularly the limited number of examples available for minority intents.

Failure Analysis - Top 5 Deficiencies

Failure 1: Connectivity Issues Misclassified as Battery Issues

Example:

“My iPhone keeps disconnecting from Wi-Fi”

System output: `battery_issue`

Expected: `connectivity_issue`

Hypothesis: The classifier relies heavily on lexical overlap and has limited labeled examples for distinguishing device-related issues. Connectivity-specific language is not sufficiently separated from broader device/battery patterns.

Failure 2: Account Security Misclassified as iCloud/Data Issues

Example:

“My Apple ID password isn't working and I think someone may have accessed my account.”

System output: `icloud_data_issue`

Expected: `account_security_issue`

Hypothesis: `account_security_issue` has only 3 labeled examples, which is insufficient for the classifier to learn

reliable security-related patterns.

Failure 3: High Retrieval Similarity Does Not Always Mean Useful Evidence

Example: “Same here!!”

System behavior: The retrieval system can assign a relatively high similarity score despite the message containing almost no information about the underlying issue.

Hypothesis: TF-IDF measures lexical similarity rather than semantic relevance or resolution usefulness. Short and generic messages are therefore particularly difficult for retrieval.

Failure 4: Generated Replies Can Add Unsupported Troubleshooting

Example:

For a battery-related query, the generated reply suggested checking battery usage and changing settings even though those specific troubleshooting steps were not present in the retrieved AppleSupport responses.

System behavior: The reply remained professional and plausible, but included information beyond the retrieved evidence.

Hypothesis: The LLM's pretrained troubleshooting knowledge can override the desired evidence-grounded behavior. Prompting reduces this tendency but does not guarantee strict grounding.

Failure 5: iOS Update Issues Misclassified as General Device Issues

Example: “Ever since I updated my iPhone, it has been freezing and becoming really slow.”

System output: device_issue

Expected: ios_update_issue

Hypothesis: The classifier focuses on the immediate symptom (“freezing”, “slow”) rather than the explicit causal context (“since I updated”). Because both intents contain general device-problem language, the lexical model struggles to prioritize the update-related context.

What Is Misleading About My Headline Number?

The headline result is 38.68% mean 5-fold accuracy, but this number can easily be misunderstood as the accuracy of the complete support agent. It is not.

First, it measures only intent classification, not whether the retrieved historical examples are relevant, whether the generated reply is helpful, or whether the escalation decision is correct. Second, the classifier was evaluated on a relatively small 181-example labeled set with significant class imbalance; some intents have only 3–6 examples, making their individual performance less reliable. Finally, the reported score comes from cross-validation, while the production classifier is retrained on all 181 examples, so 38.68% is an estimate of classifier generalization rather than a held-out end-to-end agent score.

The retrieval results also need caution: a high TF-IDF similarity score does not necessarily mean the retrieved conversation contains the right resolution.

38.68% is the mean cross-validated accuracy of a simple intent classifier on a small, imbalanced labeled dataset, not the accuracy of the complete support agent.

The more important result is therefore not a single headline number, but understanding where each component succeeds, where it fails, and why it fails!

What I'd Do with one more week

1. Expand the labeled dataset, especially minority intents and difficult boundary cases. The current classifier is limited more by data coverage than model complexity.
2. Replace TF-IDF retrieval with semantic retrieval using sentence embeddings, then compare it against the current TF-IDF approach on the same evaluation set.
3. Make retrieval quality part of the escalation decision. When historical evidence is weak or generic, the agent should prefer escalation rather than generate detailed troubleshooting from limited evidence.
4. Strengthen response grounding. Add a verification step that checks whether important troubleshooting claims in the generated reply are supported by the retrieved AppleSupport responses.
5. Evaluate the complete agent on a larger held-out set. Measure intent, retrieval relevance, reply quality, and escalation together instead of relying mainly on component-level metrics.
6. Tune escalation thresholds using validation data and explicit risk trade-offs, prioritizing avoiding incorrect automated responses over maximizing automation.

Decision Log

- **Selected AppleSupport as the target brand** - It had enough direct customer-support pairs and a broad range of support issues to support both classification and historical retrieval.
- **Defined 12 intents instead of using the dataset's raw topics** - A smaller, task-specific taxonomy makes the classification problem measurable while still covering the main AppleSupport issues.
- **Kept the natural class imbalance** - I did not artificially balance the labeled set because the imbalance reflects the sampled support traffic and exposes a real weakness of the classifier.
- **Used TF-IDF + Logistic Regression for classification** - It is simple, fast, interpretable, and provides a strong baseline without adding unnecessary model complexity.
- **Used balanced class weights** - This gives minority intents more influence during training without duplicating or synthetically generating examples.
- **Used TF-IDF + cosine similarity for historical retrieval** - The method is lightweight and makes it easy to inspect exactly which past conversations influenced a response.
- **Retrieved the top 3 historical conversations** - Three examples provide enough context for generation while keeping the evidence set small and inspectable.
- **Treated similarity as a diagnostic, not a relevance score** - High lexical similarity does not guarantee that the retrieved response is actually useful or correct.
- **Used the LLM primarily for response generation, not intent classification** - This keeps the intent evaluation reproducible and separates deterministic classification from generative behavior.
- **Grounded generation in retrieved historical responses** - The goal was to reproduce the brand's historical support behavior rather than generate generic troubleshooting from the model's prior knowledge.
- **Used classifier confidence and retrieval similarity for escalation** - Either signal alone can be misleading; combining them provides a simple evidence-and-confidence-based safety check.
- **Kept intermediate decisions visible in the CLI** - Showing intent, confidence, retrieved examples, reply, and escalation reason makes failures easier to diagnose instead of hiding everything behind a single final answer.